CampusX — 100 Days of Machine Learning

Complete Theory Notes · Day 13 to Day 20

Based on the official YouTube playlist by Nitish Singh (CampusX) · playlist: PLKnIA16_Rmvbr7zKYQuBfsVkjoLcJgxHH

Day	Topic	Video ID
13	End to End Toy ML Project	dr7z7a_8lQw
14	How to Frame an ML Problem (Detailed)	A9SezQlvakw
15	Working with CSV Files	a_XrmKlaGTs
16	Working with JSON and SQL	fFwRC-fapIU
17	Fetching Data from an API	roTZJaxjnJc
18	Fetching Data via Web Scraping	8NOdgjC1988
19	Understanding Your Data (Descriptive Stats)	mJlRTUuVr04
20	EDA using Univariate Analysis	4HyTlbHUKSw

DAY 13 — End to End Toy ML Project

youtu.be/dr7z7a_8lQw

Purpose of this Day

- Before diving deep into individual techniques, Nitish Sir builds a **complete mini ML project** from scratch to show the full workflow in action.
- Dataset used: **Placement Dataset** — student CGPA and IQ score as features, Placement (0/1) as target.
- Task type: **Binary Classification** (placed or not placed). Algorithm: **K-Nearest Neighbors (KNN)**.

The 5-Step sklearn Workflow — Applies to Every Algorithm

- **Step 1 — Load and inspect the data:**

```
import pandas as pd df = pd.read_csv('placement.csv') print(df.head()) print(df.shape) # (rows, cols)
print(df.info()) # dtypes + nulls
```

- **Step 2 — Separate features (X) and target (y):**

```
X = df[['cgpa', 'iq']] # input features (2D) y = df['placement'] # target label (1D)
```

- **Step 3 — Train/Test Split:**

```
from sklearn.model_selection import train_test_split X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, random_state=42 ) # test_size=0.2 → 20% test, 80% train # random_state=42 →
reproducible split
```

- **Step 4 — Instantiate, fit, predict:**

```
from sklearn.neighbors import KNeighborsClassifier knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train) # training y_pred = knn.predict(X_test) # prediction
```

- **Step 5 — Evaluate:**

```
from sklearn.metrics import accuracy_score print(accuracy_score(y_test, y_pred)) # e.g., 0.87
```

- **This 5-step pattern is universal in sklearn** — every algorithm (Logistic Regression, Decision Tree, SVM, etc.) follows exactly the same structure. Only step 4 changes.
- The key sklearn methods: **.fit(X_train, y_train)** — trains the model. **.predict(X_test)** — generates predictions. **.score(X_test, y_test)** — evaluates directly.

Key Concepts Introduced

- **train_test_split:** Randomly splits dataset. Never train and evaluate on same data — model would just memorize.
- **test_size:** Fraction of data held out for testing. Common: 0.2 (20%) or 0.25.
- **random_state:** Seeds the random number generator — ensures same split every run (reproducibility).
- **accuracy_score:** (Correct predictions) / (Total predictions). Only appropriate when classes are balanced.

The universal sklearn pattern:

Import → Instantiate → fit() → predict() → evaluate(). Once you know this, you can use any of sklearn's 30+ algorithms without learning new syntax. Only the class name and hyperparameters change.

DAY 14 — How to Frame an ML Problem (Detailed)

youtu.be/A9SezQlvakw

Types of Data in Machine Learning

Data Category	Sub-type	Description	Examples
Numerical	Continuous	Can take any value in a range — infinite possible values	Height, Temperature, Price, Salary
Numerical	Discrete	Countable values only — integers	Number of clicks, Students in class, Age (int)

Categorical	Ordinal	Categories with a meaningful order/ranking	Ratings (1-5), Education (UG/PG/PhD), Size (S/M/L)
Categorical	Nominal	Categories with NO natural order	Gender, City, Color, Blood group
Unstructured	Text	Raw natural language — needs NLP preprocessing	Reviews, Tweets, News articles
Unstructured	Image	Pixel arrays — needs CNN or similar	Medical scans, Photos, CCTV frames
Unstructured	Audio/Video	Time-series signals — needs special processing	Call recordings, YouTube videos

Structured vs Unstructured Data

- **Structured data:** Has a well-defined schema — rows and columns (tabular). Can be stored in a SQL database or CSV. Traditional ML algorithms (Linear Regression, Decision Tree, XGBoost) work natively.
- **Unstructured data:** No predefined format. Makes up ~80% of real-world data. Requires Deep Learning (CNN for images, Transformer for text) or specialized feature extraction first.
- **Semi-structured:** Has some organization but not fully tabular. Examples: JSON, XML, HTML. Requires parsing before use.

Framing the Output — Choosing the Right Task

- **Regression** → output is a continuous number. Ask: 'Can the output be any value on a scale?'
  - Salary prediction, house price, temperature forecast, age estimation
- **Classification** → output is one of a fixed set of classes.
  - Binary: 2 classes (spam/ham, fraud/not fraud, placed/not placed)
  - Multi-class: 3+ classes (digit 0-9, flower species, disease type)
  - Multi-label: one sample belongs to multiple classes simultaneously (a movie tagged as Action + Comedy + Thriller)
- **Clustering** → no output label exists. Goal: discover natural groupings in unlabelled data.
- **Ranking** → output is a relative ordering. Search engines, recommendation systems.

Business Metric vs ML Metric

- **ML Metric:** What the model optimizes — accuracy, F1-Score, RMSE, AUC-ROC.
- **Business Metric:** What the company actually cares about — revenue, customer retention, conversion rate.
- The two are NOT always aligned. A model with 99% accuracy on fraud detection might still miss the top 1% of high-value frauds, costing millions.
- Always define BOTH before starting a project. ML metric is the proxy; business metric is the truth.

Key insight for interviews:

When asked to design an ML system, explicitly state: (1) Data type (structured/unstructured, numerical/categorical), (2) Task type (regression/classification/clustering), (3) ML metric (e.g., F1), (4) Business metric (e.g., reduce fraud losses by 30%). Interviewers want to see you distinguish between what the model measures and what the business cares about.

DAY 15 — Working with CSV Files

youtu.be/a_XrmKlaGTs

What is a CSV?

- **CSV (Comma Separated Values)** — the most common data format in ML. Each row = one sample; columns = features; values separated by a delimiter (default: comma).
- A CSV is just a plain text file. It has no special encoding for data types — everything is a string until pandas interprets it.

Reading CSV — pd.read_csv() Key Parameters

Parameter	Purpose	Example
filepath_or_buffer	Path to file (local) or URL (remote)	pd.read_csv('data.csv') or pd.read_csv('https://...')
sep / delimiter	Column separator character	sep='\t' for TSV, sep=';' for semicolon-delimited

header	Row number to use as column names	header=0 (default), header=None if no header row
index_col	Column to use as row index	index_col='student_id' or index_col=0
usecols	Load only specific columns — saves memory	usecols=['name','age','salary']
nrows	Read only first N rows	nrows=1000 (useful for large files)
skiprows	Skip N rows from top	skiprows=2 (skip metadata rows)
na_values	Strings to treat as NaN	na_values=['NA','?','--','missing']
dtype	Force column data types on read	dtype={'age': int, 'price': float}
encoding	File encoding — crucial for Hindi/multilingual	encoding='utf-8' or encoding='latin-1'

Writing CSV

```
df.to_csv('output.csv', index=False) # index=False → don't write row numbers df.to_csv('output.csv', sep='|') # custom separator df.to_csv('output.csv', columns=['a','b']) # only specific columns
```

Common Issues and Fixes

- **UnicodeDecodeError:** File has non-UTF-8 characters. Fix: try encoding='latin-1' or encoding='cp1252'.
- **Wrong separator:** Data all in one column. Fix: specify sep='\t' or sep=';'.
- **Mixed types in column:** Pandas infers int but some cells have strings. Fix: dtype={'col': str} or low_memory=False.
- **Large file:** Use nrows or chunksize for reading in batches:

```
for chunk in pd.read_csv('big.csv', chunksize=10000): process(chunk) # process each chunk independently
```

Rule of thumb:

Always check df.head(), df.shape, df.info() immediately after loading any CSV. These three commands tell you: what the data looks like, how many rows/cols, and whether dtypes were inferred correctly.

DAY 16 — Working with JSON and SQL

youtu.be/fFwRC-fapIU

JSON — JavaScript Object Notation

- **JSON** is a text-based format for structured data, built around key-value pairs. It's the universal format for APIs and web services.
- **JSON structure** maps to Python naturally: JSON object → Python dict, JSON array → Python list, JSON string → str, JSON number → int/float.

```
# JSON example {"name": "Yash", "cgpa": 8.5, "skills": ["Python", "ML"]} import json with open("data.json") as f: data = json.load(f) # dict or list df = pd.DataFrame(data) # if data is a list of dicts
```

- **pd.read_json():** Directly reads a JSON file/URL into a DataFrame.

```
df = pd.read_json("data.json") # For nested JSON: use json_normalize from pandas import json_normalize df = json_normalize(data, record_path=["results"])
```

SQL with pandas — SQLite Workflow

- **SQLite** is a lightweight file-based SQL database. No server needed — just a .db file. Perfect for ML data workflows.
- Workflow: connect to DB → run SQL query → get result as DataFrame.

```
import sqlite3 import pandas as pd # Connect conn = sqlite3.connect('students.db') # Query → DataFrame df = pd.read_sql_query('SELECT * FROM students WHERE cgpa > 7.5', conn) # Write DataFrame → SQL table df.to_sql('students', conn, if_exists='replace', index=False) conn.close()
```

Key SQL Commands Recap

SQL Statement	Purpose
SELECT col1, col2 FROM table	Fetch specific columns
SELECT * FROM table WHERE age > 25	Filter rows with condition
SELECT * FROM table ORDER BY salary DESC	Sort results
SELECT dept, AVG(salary) FROM t GROUP BY dept	Aggregate (group and summarize)
SELECT * FROM t1 JOIN t2 ON t1.id = t2.id	Combine two tables on a key
SELECT * FROM table LIMIT 100	Get first N rows only

JSON vs CSV vs SQL — When to use what:

CSV: flat tabular data, easy to share and open. JSON: API responses, nested/hierarchical data, web services. SQL: structured data with relationships, querying subsets efficiently, production databases.

DAY 17 — Fetching Data from an API

youtu.be/roTZJaxjnJc

What is an API?

- **API (Application Programming Interface)** — a contract that lets two applications communicate. In data science, APIs are how you fetch live, real-time data (stocks, weather, social media, etc.)
- **REST API** — most common type. Works over HTTP. You send a request to a URL endpoint, get a JSON response.
- **HTTP Methods:** GET (read data), POST (send data), PUT (update), DELETE (remove). In data science, you'll mostly use GET.
- **Status Codes:** 200 = OK, 400 = Bad Request, 401 = Unauthorized (wrong API key), 404 = Not Found, 500 = Server Error.

The requests Library

```
import requests # Basic GET request response = requests.get('https://api.example.com/data')
print(response.status_code) # 200 = success print(response.json()) # parse JSON response to Python dict
```

API with Authentication — Headers and Keys

```
# Most real APIs require an API key API_KEY = 'your_api_key_here' URL =
'https://api.openweathermap.org/data/2.5/weather' params = {'q': 'Mumbai', 'appid': API_KEY, 'units':
'metric'} response = requests.get(URL, params=params) data = response.json() print(data['main']['temp']) #
e.g., 31.5 (Celsius)
```

JSON Response → DataFrame

```
import pandas as pd response = requests.get('https://api.example.com/movies') data = response.json() #
usually a list of dicts df = pd.DataFrame(data) # direct conversion if flat JSON print(df.head())
```

- **Pagination:** APIs often return results in pages (e.g., 100 per page). Use a loop to fetch all pages.
- **Rate Limiting:** Most APIs limit requests per minute/hour. Use time.sleep() between requests to avoid getting blocked.
- **API Key Security:** Never hardcode API keys in your code. Use environment variables or .env files.

Common real-world APIs for ML datasets:

OpenWeatherMap (weather), TMDB (movies), NewsAPI (news articles), Twitter/X API (tweets), Alpha Vantage (stock prices), Nutritionix (food data), CoinGecko (crypto prices). Many are free with registration.

DAY 18 — Fetching Data via Web Scraping

youtu.be/8NOdgjC1988

What is Web Scraping?

- **Web Scraping** — automatically extracting data from websites by parsing their HTML source code.
- Used when: no API is available, data is on a public website, manual copy-paste is not feasible.
- **Libraries:** requests (download HTML), BeautifulSoup (parse HTML), Selenium (for JS-rendered pages).
- **Ethics:** Always check robots.txt (e.g., site.com/robots.txt) to see what's allowed. Respect rate limits. Don't scrape private/personal data.

The requests + BeautifulSoup Workflow

```
import requests from bs4 import BeautifulSoup import pandas as pd # Step 1: Download the page HTML url = 'https://example.com/products' response = requests.get(url) html = response.text # Step 2: Parse with BeautifulSoup soup = BeautifulSoup(html, 'html.parser') # Step 3: Find elements title = soup.find('h1').text # first tag all_prices = soup.find_all('span', class_='price') # all matching tags # Step 4: Extract text and build list data = [] for item in soup.find_all('div', class_='product'): name = item.find('h3').text.strip() price = item.find('span', class_='price').text.strip() data.append({'name': name, 'price': price}) # Step 5: DataFrame df = pd.DataFrame(data)
```

BeautifulSoup Key Methods

Method	Purpose	Example
soup.find(tag)	First matching tag	soup.find('h2')
soup.find_all(tag)	All matching tags → list	soup.find_all('li')
tag.text / tag.get_text()	Get visible text content	soup.find('p').text
tag.get('attribute')	Get attribute value	tag.get('href') for links
soup.find(tag, class_='x')	Filter by CSS class	soup.find('div', class_='card')
soup.find(tag, id='x')	Filter by HTML id	soup.find('div', id='main')
soup.select('div.card > h2')	CSS selector (more powerful)	nested element selection

Quick Method — pd.read_html() for Tables

```
# If the website has tags, read all tables in one line tables = pd.read_html('https://en.wikipedia.org/wiki/List_of_countries') df = tables[0] # first table on the page
```

API vs Web Scraping — Which to use:

Always prefer an official API if one exists — it's reliable, structured, and legal. Use web scraping only when no API exists. pd.read_html() is the quickest option when the site has proper HTML tables.

DAY 19 — Understanding Your Data — Descriptive Statistics

youtu.be/mJlRTUuVr04

First Look at Any Dataset — 5 Essential Commands

```
df.shape # (rows, cols) → size of dataset df.head(10) # first 10 rows df.info() # column names, dtypes, non-null counts df.describe() # stats for numerical columns (count, mean, std, min, quartiles, max) df.isnull().sum() # count of missing values per column
```

Measures of Central Tendency

Measure	Definition	pandas Code	When to Use
Mean (Average)	Sum of values / count. Sensitive to outliers.	df['col'].mean()	Symmetric distributions, no outliers
Median	Middle value when sorted. Robust to outliers.	df['col'].median()	Skewed data, data with outliers (e.g., salary)
Mode	Most frequently occurring value.	df['col'].mode()[0]	Categorical data, discrete data

Measures of Dispersion (Spread)

Measure	Definition	pandas Code
Variance	Average squared deviation from the mean. Units squared.	df['col'].var()
Std Deviation	Square root of variance. Same units as data.	df['col'].std()
Range	Max - Min. Highly sensitive to outliers.	df['col'].max() - df['col'].min()
IQR	Q3 - Q1. Interquartile range. Robust spread measure.	df['col'].quantile(0.75) - df['col'].quantile(0.25)
Percentile	Value below which X% of observations fall.	df['col'].quantile(0.9) # 90th percentile

Shape of Distribution — Skewness and Kurtosis

- **Skewness** — measures asymmetry of the distribution.
  - Skewness ≈ 0 → symmetric (normal distribution). Mean ≈ Median ≈ Mode.
  - Positive Skew (Right Skew, skewness > 0) → tail on the right → Mean > Median > Mode. E.g., income, house prices.
  - Negative Skew (Left Skew, skewness < 0) → tail on the left → Mean < Median < Mode.
  - pandas: df['col'].skew()
- **Kurtosis** — measures the 'peakedness' and tail heaviness of a distribution.
  - High kurtosis → sharp peak + heavy tails → more extreme outliers than a normal distribution.
  - Low kurtosis → flat peak + thin tails → fewer outliers.
  - Normal distribution has kurtosis = 3 (or 0 in 'excess kurtosis'). pandas: df['col'].kurt()

Other Useful Commands

```
df['col'].value_counts() # frequency of each category (categorical cols) df['col'].nunique() # number of unique values df.corr() # correlation matrix (Pearson by default) df.duplicated().sum() # number of duplicate rows df.dtypes # data types of all columns
```

When mean ≠ median, always prefer median for central tendency.

Outliers pull the mean but don't affect median. Example: 10 employees earn ■30K/month, CEO earns ■10L/month. Mean salary = misleading. Median salary = representative of typical employee.

DAY 20 — EDA using Univariate Analysis

youtu.be/4HyTlbHUKSw

What is EDA and Univariate Analysis?

- **EDA (Exploratory Data Analysis)** — the process of visually and statistically exploring your data BEFORE modelling to understand its structure, patterns, and problems.
- **Univariate Analysis** — analysing ONE variable at a time. Goal: understand its distribution, central tendency, spread, and outliers.
- EDA types: Univariate (1 variable), Bivariate (2 variables), Multivariate (3+ variables). Day 20 covers Univariate.

Univariate Analysis — Numerical Features

Plot	What it Shows	Code
Histogram	Distribution shape, frequency of value ranges	plt.hist(df['age'], bins=20) or sns.histplot(df['age'], kde=True)
KDE Plot	Smooth continuous estimate of distribution	sns.kdeplot(df['salary'])
Box Plot	Median, IQR, whiskers, and outlier points	sns.boxplot(y=df['col']) or plt.boxplot(df['col'])
Violin Plot	Combines KDE + boxplot — shows full distribution	sns.violinplot(y=df['col'])
describe()	Count, mean, std, min, Q1, Q2 (median), Q3, max	df['col'].describe()

Reading a Box Plot — The 5-Number Summary

- **Q1 (25th percentile):** 25% of data falls below this value. = bottom of the box.
- **Q2 / Median (50th percentile):** Middle of the data. = line inside the box.
- **Q3 (75th percentile):** 75% of data falls below this. = top of the box.
- **IQR = Q3 - Q1:** Width of the box. Larger box = more spread.
- **Whiskers:** Extend to Q1 - 1.5×IQR (lower) and Q3 + 1.5×IQR (upper).
- **Points beyond whiskers = outliers** — plotted as individual dots.

Univariate Analysis — Categorical Features

Plot	What it Shows	Code
Count Plot / Bar Chart	Frequency of each category	sns.countplot(x='gender', data=df)
Pie Chart	Proportion of each category (use sparingly)	df['col'].value_counts().plot(kind='pie')
value_counts()	Exact count + frequency of each category	df['col'].value_counts(normalize=True) # proportions

What to Look for During Univariate EDA

- **Distribution shape:** Is it normal (bell curve)? Skewed? Bimodal (two peaks — suggests two subgroups)?
- **Outliers:** Extreme values visible in boxplot or far right/left of histogram. Need to decide: keep, cap, or remove?
- **Missing values:** Does the distribution tell you WHY values are missing (e.g., low-income earners don't report salary)?
- **Data range:** Are there impossible values? (age = -5, salary = 0 for an employed person).
- **Cardinality of categoricals:** Too many unique values in a categorical column = high cardinality = encoding challenges.
- **Class imbalance:** For the target variable, if one class dominates (e.g., 95% Not Fraud), accuracy is misleading — need F1 or AUC.

```
import matplotlib.pyplot as plt import seaborn as sns # Full univariate EDA template for numerical col col = 'salary' fig, axes = plt.subplots(1, 3, figsize=(15, 4)) axes[0].hist(df[col], bins=30) # histogram sns.boxplot(y=df[col], ax=axes[1]) # boxplot sns.kdeplot(df[col], ax=axes[2]) # KDE plt.suptitle(f'Univariate Analysis: {col}') plt.show() print(df[col].describe())
```

EDA golden rule:

EDA is not optional — it IS the most important step. A good EDA prevents you from feeding garbage into your model. Common EDA discoveries: wrong dtypes, hidden missing values coded as '?', outliers that are actually data entry errors, and class imbalance in the target column.

Important Things From My Side

Concepts not explicitly in the videos but essential to know alongside Days 13–20.

1. train_test_split — Why 80/20 and Why random_state Matters

- The 80/20 split is a convention, not a rule. For small datasets (<1000 rows), prefer 90/10. For very large datasets (1M+ rows), even 99/1 works.
- **Stratified split:** When your target is imbalanced (e.g., 95% Not Fraud), a random split might put all fraud cases in training. Fix: stratify=y.

```
X_train, X_test, y_train, y_test = train_test_split( X, y, test_size=0.2, random_state=42, stratify=y # preserves class proportions )
```

- **Data leakage:** Never fit transformers (like StandardScaler) on the full dataset. Fit ONLY on training data, then transform both train and test.

```
from sklearn.preprocessing import StandardScaler scaler = StandardScaler() scaler.fit(X_train) # fit ONLY on train X_train = scaler.transform(X_train) X_test = scaler.transform(X_test) # transform test using train's stats
```

2. Correlation vs Causation

- df.corr() gives the Pearson correlation coefficient between numerical columns. Range: -1 (perfect negative) to +1 (perfect positive). 0 = no linear relationship.
- **Correlation does NOT imply causation.** Ice cream sales correlate with drowning deaths — both are caused by hot weather (confounding variable). A model trained on correlations might look accurate but be completely wrong in production.
- Use correlation for feature selection: drop features with near-zero correlation to target. Drop one from a highly correlated pair (multicollinearity).

3. Handling Missing Values — Preview (will come in later days)

Technique	When to Use
Drop rows (df.dropna())	When very few rows have missing values (<5%)
Mean imputation (SimpleImputer(strategy='mean'))	Numerical data, roughly symmetric distribution
Median imputation	Numerical data with outliers or skewed distribution
Mode imputation	Categorical columns
KNN Imputer	When missing values have patterns — uses neighbor values

4. Feature Scaling — Why it Matters for KNN

- KNN uses distance to find neighbors. If one feature is in thousands (salary) and another is 0-1 (score), salary will dominate the distance calculation — the model effectively ignores the score feature.
- **Standardization (StandardScaler):** z = (x - mean) / std. Output: mean=0, std=1. Works for most algorithms.
- **Normalization (MinMaxScaler):** x_norm = (x - min) / (max - min). Output: [0, 1]. Sensitive to outliers.
- Rule: **Always scale before KNN, SVM, and Neural Networks.** Tree-based models (Decision Tree, Random Forest, XGBoost) don't need scaling.

5. The Difference Between df.describe() and df.info()

Command	What it shows	Best for
df.info()	Column names, non-null count, dtype for every column. Shows memory usage.	Checking missing values and data types
df.describe()	For numerical cols: count, mean, std, min, 25%, 50%, 75%, max.	Understanding numerical distributions
df.head(n)	First n rows of the DataFrame.	Visual sanity check of raw data
df.shape	(rows, cols) tuple.	Quick size check
df.nunique()	Number of unique values per column — all columns.	Spotting high-cardinality or near-constant columns

End of Day 13–20 Complete Theory Notes · CampusX 100 Days of Machine Learning

After completing Days 21–22, come back for the next batch covering Bivariate & Multivariate Analysis and Pandas Profiling.